目录

2026-10 上旬调度子系统：重点补丁与特性分析 · · · · · 2
01 · 焦点：lost wakeup 修复合入 tip/sched/urgent · · · · · 2
02 · fair push 机制：短 slice 任务时延的体系化解决 · · · · · 4
03 · EEVDF slice 保护（vprot）不变量链 · · · · · 5
04 · CFS 杂项：task_h_load 重做 与 sched_setparam 快路径 · · · · · 6
05 · Cache-aware 调度打磨潮（四条线并行） · · · · · 7
06 · sched/numa：fix several NUMA balancing issues（0/6） · · · · · 9
07 · NUMA 方向讨论：距离矩阵 与 粒度控制之争 · · · · · 11
08 · sched_ext：NUMA 支持、修复潮与 sub-scheduler 生态 · · · · · 11
09 · 基础设施与观测性 · · · · · 13
附 · 正确性修复简报 · · · · · 14
结 · 收尾观察 · · · · · 15

2026-10 上旬调度子系统：重点补丁与特性分析

> 数据来源：本仓库 `sched/2026/10/` 下 10-01 ~ 10-14 共 **187 篇**日报邮件（去重后 104 个逻辑系列）。深析条目均引用补丁关键代码与邮件原始数据；纯正确性小修复在附录简报。文中 lore 链接均为邮件真实 Message-ID 直链。

- **104** 逻辑系列
- **20** 已进 tip / 主线
- **~50** 评审中
- **15** 本文深析条目

① EEVDF 语义收尾

slice 保护（vprot）不变量修复链收口、min-slice 选核、层级负载重做——EEVDF 从”能跑”走向”边界正确”。

② 放置策略体系化

fair class 首次获得 push 迁移机制；cache-aware 调度进入正确性 + 开销打磨期，多条修复具备 stable 候选资格。

③ sched_ext 生态成型

v7.3-rc6 修复批合入主线；sub-scheduler（父子调度器）委派契约、caps 通知、NUMA 支持相继落地。

01 · 焦点：lost wakeup 修复合入 tip/sched/urgent

作者 **K Prateek Nayak**（AMD） · 报告 **Paul E. McKenney** · 合入 **tip/sched/urgent** · **[merged · 118a2cf4ced6]** · 日报 sched-20261013-001 / 20261014-001

背景与问题

rcutorture TREE01 场景下调度器「丢失唤醒」：CPU 带着一个完全可运行的 RT 任务却陷入 idle，触发 RCU CPU stall。dump 出的 rq 状态自相矛盾——等待任务是 RT，但 `rq->next_class` 还停在 fair：

stall 时刻的 rq 状态（tip commit message）（`kernel/sched/core.c`）

```
rq->next_class      = &fair_sched_class
rq->nr_running      = 1
rq->cfs.h_nr_queued = 0
rq->rt.rt_nr_running = 1
```

根因是 `sched_change_end()` 与 `__sched_setscheduler()` 的竞态：`sched_balance_newidle()` 临时 drop rq_lock 的窗口内，任务被 enqueue 后又从 fair 升到 RT；此时 `ctx->queued` 为真、`ctx->running` 为假，`next_class` 升级与 `resched_curr()` 全被跳过，newidle 平衡只能选 idle：

竞态时序（节选自 commit 118a2cf4ced6）（`race`）

```
  CPU0                                  CPU1
__schedule()
  pick_task_fair()
    sched_balance_newidle()
      raw_spin_rq_unlock(rq0)
                                rq_lock(rq0)
                                activate_task(rq0, p)   /* p is a fair task */
                                __sched_setscheduler(p)
                                  /* p is moved to SCHED_RR */
                                  sched_change_end()
                                    /* p->on_rq = 1 (ctx->queued)
                                     * but p is not rq->donor (!ctx->running) hence
                                     * resched_curr() and rq->next_class upgrade
                                     * is skipped. */
        raw_spin_rq_lock(rq0)
         if (rq->cfs.h_nr_queued && ...)      /* false */
         if (rq_modified_above(...))           /* false: next_class 仍为 fair */
    return NULL;  /* pick_task_fair() */
  pick_task_idle()
!!! rq->idle 带着 rq->nr_running == 1 切入，但 !need_resched() !!!
```

方案思路

把「升级检测 + 唤醒抢占」从 `ctx->running` 分支拆出：对已入队（未运行）任务在改类后也调用 `wakeup_preempt()` 重新评估 `rq->next_class` 并置 NEED_RESCHED。改动仅 `kernel/sched/core.c` +9/−10：

修复 diff（最终合入版）（`kernel/sched/core.c`）

```
        if (p->sched_class->switched_to)
            p->sched_class->switched_to(rq, p);

+       /*
+        * Check if the task can preempt the current in case the class
+        * was upgraded. Note that none of the switch*_from() methods
+        * know the new class and none of the switch*_to() methods know
+        * the old class so we must resched here.
+        */
+       if (ctx->queued)
+           wakeup_preempt(rq, p, 0);
+
        if (ctx->running) {
```

Fixes 标签经作者纠偏为 `704069649b5b`（“Rework sched_class::wakeup_preempt() and rq_modified_*() “）——该 commit 之前 enqueue 时改的是 `queue_mask`、能捕获类变更，改走 `next_class` 路径后才引入此洞。

效果

- **> 2 个数量级** — bug 率下降（Paul，99.9% 置信）
- **27.5–30.2 GP/s** — 512 CPU · 64×TREE01 各实例
- **exitcode 0** — 加 diff 后无 complaint

10-13 发出 → Peter「W00t, ok, let me go queue this stuff!」→ 10-14 合入 tip/sched/urgent，Review/Tested 标签（Vincent、Phil Auld、Paul）当日闭环。社区下一步关注是否需要 `Cc: stable` 回合。

lore：补丁 · bug 报告根 · tip commit

02 · fair push 机制：短 slice 任务时延的体系化解决

作者 **Vincent Guittot**（Linaro）· **18 补丁 v2**（10-02 发，10-09~14 密集 review）· **[under review · v3 在望]** · 日报 sched-20261009-004 / 20261010-001 / 20261013-013

背景与问题

短 slice 任务（小忙小睡、频繁唤醒的时延敏感任务）在 EEVDF 下有两类系统性时延：**放置阶段**——选核不看候选 CPU 的 min_slice、`wake_affine` 不比较 slice、唤醒后 lag 未清，任务落在「最快能跑」的 CPU 之外；**排队阶段**——CFS 没有 RT 式 push 机制，任务在某 CPU 排不上队首时只能干等本地调度事件。EAS 的 `feec()` 选核同样不看 slice。

方案思路：18 补丁三块

- **EEVDF 基础（01–05）**：休眠实体正 lag 衰减、idle CPU 唤醒 reset lag、per-CPU 缓存 min_slice、wake_affine 比较 min slice、选核加 min slice check；
- **fair push 机制（06–14）**：默认关闭的 `sched_push_task` 静态分支；关键设计是职责分离——`fair_check_pushable_task()` 做轻量、不持 pi_lock 的 best-effort 检查，真正的「迁到哪个 CPU」交给 `select_task_rq_fair()`；force-push 变体；PREEMPT_SHORT 短任务是第一个用户；
- **EAS 整合（15–18）**：`feec()` 从 spare capacity 改用 OPP cost，选核把任务 slice 与目标核 min_slice 比较。

最重要的取舍：push 做成**通用基础设施**而非短任务专用。Chen Yu 已按 Qais 建议做了 cache-aware 复用 push 的评估补丁（+82/−7）——cache-aware 敏感任务经 push 迁到偏好 LLC；论点是 push 触发门槛（需抢占 runnable task）比 wakeup 高、频次低，可减少与 cache-aware 负载均衡器的竞争、避免 wakeup 路径观察到的 task bouncing。

评审抓出的代表性问题——Tim Chen 指出 push 复用 `select_idle_sibling()` 会**污染 `recent_used_cpu` 唤醒提示**（push 的 prev 是「当前排队 CPU」而非「上次运行 CPU」，失败后 hint 退化、后续唤醒丢失第二候选），修复是给该函数加 `select_flags` 参数、仅 `WF_TTWU` 时更新：

Tim Chen 对 06/18 的修复（+7/−4，方向已被作者接受）（`kernel/sched/fair.c`）

```
+   if (!(select_flags & WF_TTWU))
+       return recent_used_cpu; /* push 语义下不更新唤醒提示 */
    p->recent_used_cpu = prev;
```

效果

实测（Chen Yu，192 核 Xeon，默认参数 hackbench）	DIFF%
process G1（最好）	+2.50%
threads G8（最差）	−7.42%
8 组结论	多数在噪声带内，“no obvious difference”

混合 slice 长度负载数据待补；cache-aware × push 评估补丁的迁移测试在途。v1 时代 min-slice 补丁在 dragonboard 上有 cyclictest/hackbench 改善（9 月数据）。

> **评估**：默认负载中性，价值在机制铺垫——push 将成为 misfit / EAS / cache-aware 的统一迁移入口。review 意见逐条被认领（Chen Yu 认领 pushable 检查瘦身），v3 在望，合入概率 medium。

lore：系列 cover · 07/18 push 机制

03 · EEVDF slice 保护（vprot）不变量链

作者 **Kayra Cizmeci** / Christian Loehle · 10-01 ~ 10-14 持续 **5 轮** · **[under review · v2]** · 日报 sched-20261001-001 → 20261014-006

背景与问题

slice protection（vprot）是**绝对 vruntime 边界**，但多条路径对其处理不一致：改任务 slice 不丢剩余 request（100us 请求却拿到 13–28ms 新鲜保护）；reweight 移动 vruntime 后「已过期」的保护会**复活**，current 越过 deadline 更早的 eligible 实体被选中；`sched_setattr()` 恢复 current 时不更新保护。Christian 的 5 补丁 v2 经 Peter 评审后解体重组——1/5 与 3+5/5 被否（破坏稳态行为），4/5 折叠进 Kayra 的独立修复由她署名重发。

方案思路：v2 彻底去掉 rel_vprot 标志

v1 用 `rel_vprot` 标志把 protection 在 reweight 时转/恢复相对量；Vincent/Prateek 质疑后，v2（10-14）改为两处协同：

reweight_eevdf()：curr 时直接以 vruntime 重建 vprot（`kernel/sched/fair.c`）

```
    if (on_rq) {
-       if (rel_vprot)
-           se->vprot += avruntime;
        se->deadline += avruntime;
        se->rel_deadline = 0;
        se->vruntime = avruntime - se->vlag;

+       if (rel_vprot)
+           se->vprot += avruntime;
+       else if (curr)
+           se->vprot = se->vruntime;
+
        if (!curr)
            __enqueue_entity(cfs_rq, se);
```

enqueue_task_fair()：按权重变化就地缩放重建（enqueue 路径）（`kernel/sched/fair.c`）

```
+   if (live) {
+       old_weight = se->h_load.weight;
+       dist = se->vprot - se->vruntime;
+   }
    reweight_eevdf(cfs_rq, se, weight, false);
    place_entity(cfs_rq, se, flags | ENQUEUE_QUEUED);
+
+   if (live) {
+       if (weight != old_weight)
+           dist = div64_long(dist * old_weight, weight);
+       se->vprot = se->vruntime + dist;
+   } else if (running) {
+       cancel_protect_slice(se);
+   }
```

效果

- **441,920 次权重变更** — 零复活、零错误选择（对照 32 次复活/7 次错选）
- **base 问题全消失** — v2 作者自测
- **3/1500** — 残留 vprot 滑动 >5%，复测归零、原因未明
- **1005→101us** — HRTICK 粒度修复数据（该补丁后被放弃）

关联回归：已合入的 `819224e506bc`（remove dead code on enqueue_task_fair）让 `reweight_eevdf()` 会被 curr 调用，与本系列在 `cfs_rq->curr` 语义上相互牵连（见附录）。

lore：v2 · v1

04 · CFS 杂项：task_h_load 重做 与 sched_setparam 快路径

task_h_load() 重做（Peter Zijlstra）[high · Vincent 全量 RB]

v2（9-29 发）· 10-14 Vincent 四补丁全量 Reviewed-by · 日报 sched-20261002-011 / 20261014-008

背景：`task_h_load()` 沿 cgroup 层级算层级负载，旧实现经 `update_cfs_rq_h_load()` 懒更新（调用方不持 rq 锁）；Peter 重做时 v1 的 back-link 遍历在 `set_next_task_fair()` 里 clobber `se`/`cfs_rq`，**192 核机器启动 panic**（Chen Yu、Vincent 双独立复现）。

方案：v2 把 back-link 跟踪移入 `for_each_sched_entity()` 本身（2/4 顺带把 `cfs_rq_of(se)` 折进去），在持锁位点重算 h_load、删除不持锁的 `update_cfs_rq_h_load()`；4/4 重做 `task_h_load()`。语义取舍经多轮澄清：root 更新频率变化有意、h_load 懒传播。

效果：正确性修复为主、无 benchmark；10-14 Vincent 全量 RB（3/4 附「back-link 或可独立成补丁」边界意见）。合入概率 high。

lore：v2 cover

sched_setparam() no-change 快路径（Chengfeng Lin）[RFC · Peter 认可重写]

日报 sched-20261009-008 / 20261013-007

背景：SCHED_OTHER 任务用默认 slice 时，遗留包装 `_sched_setscheduler()` 把 `attr.sched_runtime` 留 0（「用默认」），而 `p->se.slice` 存实际时长——两者比较必不等，**参数完全没变的 `sched_setparam()` 也走完整更新路径**。作者是在追查 v6.12.95↔v7.2 间 no-change 负载 6.5–7.1% 耗时上升时发现的。

方案：把 no-change 捷径扩展到「默认 slice 且未变」。Peter 嫌初版条件太绕，给出更简的 mode-first 比较：

Peter 建议的 mode-first 条件（v4 采纳）（`kernel/sched/core.c`）

```
    if (!!attr->sched_runtime != !!se->custom_slice)
        return true;    /* 模式切换 */
    if (attr->sched_runtime && attr->sched_runtime != se->slice)
        return true;    /* slice 变化 */
```

效果（i7-12700KF，A/B/A，15 样本 × 50 万次调用，CV < 0.30%）：

场景	Original A	Revision	Original B
Unchanged default slice	615.98 ns	162.39 ns	617.03 ns
Unchanged custom slice（对照）	162.69 ns	163.22 ns	162.45 ns

默认 slice 场景省 73.66%；作者诚实标注为合成 syscall 微基准、非应用加速。Peter 回「Yes, this looks reasonable」。

05 · Cache-aware 调度打磨潮（四条线并行）

cache-aware 调度（按 LLC 聚合任务）进入落地期，10-06~14 暴露出一组正确性与开销问题，四条修复/优化线并行推进——这是 10 月与负载均衡放置策略最相关的主题。

5.1 Honor asym packing over cache-aware（Tim Chen v2）

背景：AMD Ryzen AI HX 370 上 cache 密集的 Clang full-LTO 链接受阻——小核频率低（3.3 vs 5.1 GHz）、L3 减半（8 vs 16 MB），cache-aware 把任务钉到小核 LLC 双重受损。冲突根源：asym packing 要最高优先级 CPU，cache-aware 要共置同一 LLC、不论优先级。

方案：`can_migrate_llc_task()` 在进入「too many threads / exceed LLC capacity」检查前先判 asym packing 放行条件（v2 相对 v1 的实质改动是补上 `env->idle` 前置，Kayra 指出后采纳）：

关键放行条件（kernel/sched/fair.c +18/−3）（`guard`）

```
+   if (env->idle && sched_asym(env->sd, dst_cpu, src_cpu))
+       return 0;   /* asym packing 优先，跳过 LLC 聚合迁移 */
```

效果：Chen Yu 用 QOS 限 L3 ways + pstate 限频在 AMD 8945HX 上仿真 hybrid 实测（8 线程 pointer-chase ring、24 MB 工作集）：

配置	cache-aware ON	cache-aware OFF
补丁前	~230 M/s（6/8 线程困在小核 LLC）	~530 M/s（差距 −56.6%）
补丁后	520.7 ± 18.3 M/s（8 线程全在大核 LLC）	512.9 ± 16.6 M/s（+1.5%，噪声）

回归完全消除；Kayra RB，已 `Cc: stable # 7.2.x`。

5.2 Avoid publishing partial LLC utilization（Jemmy Wong v2）

背景：负载均衡因 all-pinned redo、或 CPU hotplug 使 `env->cpus` 与 `sched_group_span(group)` 不一致时，把只覆盖部分 CPU 的利用率当整组发布——「部分利用率 + 完整容量」让 `can_migrate_llc()` 误判忙碌 preferred LLC 有空余。

v2 折中：便宜的 redo 标志先行，子集检查兜底（+8）（`kernel/sched/fair.c`）

```
+   if ((env->flags & LBF_ALL_PINNED) ||
+       !cpumask_subset(sched_group_span(group), env->cpus))
+       return;
```

效果：无性能数字（逻辑正确性、Found by inspection）；Tim Chen 与 Chen Yu 10-14 双 RB，Tim 认定 7.2.x stable 候选。

5.3 Remove old cache group footprint on exec（Jemmy Wong）

背景：`exec` 在重置 NUMA fault 统计之前就换掉 cache group，而只有 exit 路径会减去任务的 footprint 贡献——CLONE_VM 无 CLONE_THREAD 的任务 exec 后旧贡献残留，抑制 LLC capacity 检查。**方案**（10-14 更新）：`grp->footprint` 改 `atomic_long_t`、去 clamp，placement 前快照 `total_numa_faults`、循环后结净差：

每 pass 一次更新，复用既有 NUMA 计数（`kernel/sched/fair.c`）

```
+static inline void sched_cache_footprint_update(struct task_struct *p, long delta)
+{
+   struct sched_cache_group *grp = rcu_deref_sched_cache_grp(p);
+
+   if (grp && delta)
+       atomic_long_add(delta, &grp->footprint);
+}
    /* task_numa_placement() 内 */
+   unsigned long old_faults = p->total_numa_faults;
    ... /* 遍历各节点做 placement */
+   sched_cache_footprint_update(p, p->total_numa_faults - old_faults);
```

去 clamp 的论证：footprint 聚合的正是 `total_numa_faults` 追踪的同一批贡献，NUMA placement 本就做带符号增量不 clamp；clamp 反而会掩盖记账 mismatch。failed-exec 的负值例外已论证（该组只属即将 exit 的任务）。

5.4 Reduce task_cache_work overhead（Luo Gengkun v10，华为）

背景：`task_cache_work()` 扫遍系统所有 CPU 找 pref_llc，多实例（Redis）场景开销放大。**方案**：`visited_cpus` 只扫被访问过、未超时的 CPU（超时驱逐），顺势移除 `get_scan_cpumasks()`。**效果**（AMD 服务器，valkey-benchmark 400k rps，NUMA balancing 关闭）：

- **−25.68%** — cache-aware 无此补丁的 p99 回退
- **−1.14%** — 加上 visited_cpus 后的 p99 回退（0.436→0.441ms）
- **0.81% → 0.02%** — task_cache_work 的内核态 cycles 占比
- **384 → 16** — 单次扫描的 CPU 数

lore：asym packing v2 · partial LLC v2 · exec footprint 更新

06 · sched/numa：fix several NUMA balancing issues（0/6）

作者 **Li Zhe**（字节跳动） · 10-14 v1 首发当日即收 6 封 review · **[4/6 方向被 reviewer 否定]** · 日报 sched-20261014-002

背景与问题

三类缺陷：① scan-period ratio 的分母校验与实际除数是两套 sum，memory hotplug 后 `for_each_online_node()` 跳过已下线节点，留下「local>0 但 private/shared 均为 0」的窗口——**除零**（v7.3-rc1 QEMU hot-remove NUMA 节点复现）；② 「共享 fault 应减速扫描」的实现用了 private ratio 当减速信号，共享主导窗口反而落进加速路径（2017 年以来行为）；③ memory-tiering 的 promotion fault 污染正常 task-placement 统计、free-space bypass 破坏 CXL+DRAM 刻意铺开的带宽分布。

方案思路（关键代码）

2/6：校验实际分母，修除零（`kernel/sched/fair.c`）

```
-   if (local + shared == 0 || p->numa_faults_locality[2]) {
+   if (!locality_faults || !type_faults ||
+       p->numa_faults_locality[NUMA_FAULT_MIGRATE_FAIL]) {
        p->numa_scan_period = min(p->numa_scan_period_max,
            p->numa_scan_period << 1);
```

3/6：新增「既 remote 又 private」计数器，只有它足够大才加速扫描（`kernel/sched/fair.c`）

```
-   lr_ratio = (local * NUMA_PERIOD_SLOTS) / (local + remote);
-   ps_ratio = (private * NUMA_PERIOD_SLOTS) / (private + shared);
+   lr_ratio = (local * NUMA_PERIOD_SLOTS) / locality_faults;
+   sr_ratio = (shared * NUMA_PERIOD_SLOTS) / type_faults;
+   rp_ratio = (remote_private * NUMA_PERIOD_SLOTS) / locality_faults;
+   ratio = max(lr_ratio, sr_ratio);
    } else {
+       if (rp_ratio >= NUMA_PERIOD_SLOTS - NUMA_PERIOD_THRESHOLD)
+           diff = -(NUMA_PERIOD_THRESHOLD - ratio) * period_slot;
+       else
+           diff = 0;
    }
    /* task_numa_fault() 侧记账 */
+   if (!local && priv)
+       p->numa_faults_locality[NUMA_FAULT_REMOTE_PRIVATE] += pages;
```

5/6：区分 slow-tier 元数据与真 cpupid，tiering fault 不喂 placement（`kernel/sched/fair.c`）

```
+   if (last_cpupid == LAST_CPUPID_ACCESS_TIME) {
+       if (!(sysctl_numa_balancing_mode & NUMA_BALANCING_NORMAL)) {
+           if (migrated)
+               p->numa_pages_migrated += pages;
+           return; /* 只记迁移，不喂 AutoNUMA 统计 */
+       }
+       last_cpupid = LAST_CPUPID_UNSET;
+   }
```

效果与社区态度

除 2/6 的 QEMU hotplug 复现外**无性能数据**。Gregory Price 当日对 6 个补丁逐封回帖，普遍要求补「用户可见的性能影响」；对 4/6（全局 sysctl 控 free-space bypass）明确表态：

> A global switch for this is a poor solution for this problem. ... if the user did not set MPOL_F_NUMA_BALANCING - that spread is already protected because the mempolicy overrides the numa balancing fault and no migration occurs. I don’t think this is the right direction.
>
> — Gregory Price，Re: [PATCH 4/6]

对 3/6 的担忧：「改变 2017 年以来的行为，担心意外回归，要性能数字」。合入需先补 benchmark 并重做 4/6 的接口形态。

lore：系列 cover

07 · NUMA 方向讨论：距离矩阵 与 粒度控制之争

两级亲和性打分（Jianyong Wu，海光，23 补丁 RFC）

9 月起评审，10 月三轮收敛 · **[rfc · 设计论证]** · 日报 sched-20261002-009 / 20261009-010

背景：cache-aware 负载均衡只在 LLC 粒度决策、跨节点放置交给独立的 NUMA balancing，二者互不知情；海光等非对称互连平台节点间距离非单值，需要区分度。

收敛过程：原方案造「去重距离矩阵」→ Tim Chen 多轮做减法 → 改用 **raw distance + 三元组全序** `(distance, |node_id_i − node_id_j|, min(i,j))` **平局决胜**，打分式去掉 `affinity_bias`（「bias 扭曲 affinity 得分」）→ 10-09 Chen Yu 再提分层简化：LLC 是对称的，直接把 preferred LLC 的 range 个兄弟加进 preferred_mask，**LLC 层可整体去掉距离比较**。

效果：无实测数据，纯设计论证。

NUMA balancing 粒度控制（Li Zhe vs Yafang Shao）

[方向未收敛] · 日报 sched-20261012-006 / 20261013-004 / 20261014-012

背景：全局 `numa_balancing[_mode]` 是 all-or-nothing——延迟敏感服务只要调度统计不要 tiering 提升，批处理要 PMEM 热提升，单一开关无法两全。

两条路线：Li Zhe 的 per-process prctl（K8s/容器运行时设置、子进程继承，0/4）；Yafang Shao 的 `bpf_numab_ops` struct_ops per-cgroup 策略（RFC v2）——内核跑「该 cgroup 及其所有祖先」的策略并 OR 合并，task 面传 `task_struct*`、内存面传 `folio*`，语义 **DISABLED beats NORMAL beats TIERING**（策略总能 opt-out），无策略挂载时 no-op 退化到全局 sysctl。

进展：无 benchmark；Li Zhe 10-14 表态「三元全局键可行；若维护者倾向 per-cgroup，愿意帮 review Yafang 系列」——作者立场松动，最终形态取决于维护者定调。**与 cpuset/cgroup 方向高度相关，值得跟踪参与**。

lore：Li Zhe 0/4 · Yafang BPF RFC

08 · sched_ext：NUMA 支持、修复潮与 sub-scheduler 生态

8.1 Add NUMA balancing support（Andrea Righi，NVIDIA）

5 补丁 · sched_ext/for-7.4 · **[under review]** · 日报 sched-20261004-001 → 20261010-012

背景：自动 NUMA balancing 对 sched_ext 任务**事实性关闭**——扫描只从 fair tick 排队。Vladimir Vdovin 的 KVM 宿主机实测（2 节点 160 CPU）：

- **0 / 2.4M–3.9M** — numa_pte_updates/s：SCX vs fair
- **37% / 83%** — 跑在 preferred nid 的任务比例
- **58%** — SCX 任务无 preferred_nid

方案：`SCX_OPS_NUMA_BALANCING` opt-in，扫描从 sched_ext tick 驱动（与 `task_tick_fair()` 同构）、fault 记账照常，但**任务放置留给 BPF 调度器**；preferred node 经 kfunc 暴露。10-10 接受 Tejun 建议，接口从「flag + kfunc 轮询」改为**事件式回调**：

v2 接口方向：sched_setnuma() 里、rq 锁下通知（`sched_ext ops`）

```
+   void (*set_numa_node)(struct task_struct *p, s32 nid);
/* 实现该回调即启用 hinting-fault 扫描；初始 preferred node 在 enable 时上报 */
```

效果：Vladimir 回植 6.18.54 + 自家调度器实测：4G 内存 **20 秒**完成迁移，为系列有效性背书。

lore：系列 cover

8.2 v7.3-rc6 修复批 + 后续修复潮（Tejun Heo 等）

[merged · 762122d75e5] · 日报 sched-20261007-001 等

- **GIT PULL（10-07，已进 torvalds/master）**：dying CPU 任务停在 user DSQ / BPF custody 时 `cpu_down()` 永久挂起（修复：rq 下线时重入队到本地 DSQ）；remote local DSQ 迟到的 `ops.dequeue()`；防陈旧 dispatch 的 qseq 从 rq 级改 per-task 计数器（跨 rq 迁移撞号，`Cc: stable v6.12+`）。4 commit，+528/−17 含自测。
- **ENQ_LAST idle stall**：本应交给 `ops.enqueue()` 的任务在三条 skip 路径直达 local DSQ，而 pick 已结算到 idle——CPU 挂着排队任务空转；scx_mitosis 在 mass cgroup teardown 时 exiting 任务卡 `exit_mmap()` **40+ 秒**。修复：切进任务时置 need_resched。
- **user DSQ UAF**：deferred reenqueue 可在 RCU 宽限期内对已销毁的 user DSQ 解引用。
- **proxy donors**：有 slice 剩余的 donor 留本地 DSQ 头部——经 v1→v2→Tejun 自我纠偏（“Sorry, I steered this the wrong way”）回到 PICK_PENDING 语义的三轮往返。
- **受限 CPU 集下 idle 选择不尊重任务亲和性**（Hui Su，已合入 tip）。

8.3 sub-scheduler（父子调度器）生态成型（Tejun Heo 等）

部分已合入 tip · 日报 sched-20261006-001 / 20261008-004 / 20261014-015

背景：父调度器把 cid（CPU）与 caps 委派给子调度器后存在两类盲区——**用量黑盒**（按需定尺寸的父级无法度量各子级实际用了多少 cid 时间）与**生效时机黑盒**（grant/revoke 只记录目标值，真正生效要等下一次 dispatch；子级留下的低 cpuperf 目标在 PERF 回收后残留）。

- `ops.sub_cid_sched_updated()`：cid 上的调度器变化时通知（NONE/SELF/直接子 cgroup id；子树内部变化不上报）。开销控制在代码层：调度器不变时每次上下文切换仅一次指针比较，机制整体挂 static key。
- `ops.sub_child_ecaps_updated()`：子级 caps 变化后同上下文通知直接父级；v2 砍掉 disable 报告与嵌套 dispatch 门控，改由 `sub_detach()` 恢复。配套裁决：cpuperf target 属 last-writer-wins，恢复责任归 grant 方。

· 契约易误用的实证：Tao Cui 报告「子调度器任务严重欠调度」（duty ~0.3% vs root 任务 ~100%），Tejun 定位为**使用问题**——父级从不调 `scx_bpf_sub_dispatch()`、grant 缺 `SCX_CAP_ENQ`；报告者 10-14 确认并将修正调度器复测。值得后续提议补文档/示例注释。

09 · 基础设施与观测性

SparseBitmap / SBM（K Prateek Nayak + Peter Zijlstra，13 补丁 RFC v3）

[rfc v3] · 将在 LPC'26 Scheduling MC 报告 · 日报 sched-20261002-007

背景：大容量多插槽机器上调度器/定时器的全局 cpumask（如 `nohz.idle_cpus_mask`）是所有核争写的共享 cacheline——每次置位/清位都跨 socket 抢 cacheline（C2C ping-pong）。

方案：按 LLC/节点实例**分片**的位图（每实例只表示少量 CPU，写操作留在本地 cacheline，代价是遍历更贵）。CPU 激活时动态分配索引，编码为两段 `[数组下标 | 位偏移]`，按拓扑推导 shift/mask；v3 补齐 x86 之外 7 个架构 enablement（LoongArch/MIPS/PowerPC/s390/sparc64 + 通用 arch_topology），首个用户是 nohz idle 掩码。定位：「sbm 之于 sbitmap，如同 cpumask 之于普通 bitmap」。

掩码更新代价（%cycles，AMD 内部微基准，相对全局掩码）（`benchmark`）

```
global mask                                 : 100.0000%  (var: 3.28%)
per-NUMA mask                               :  32.9209%  (var: 7.77%)
per-LLC mask                                :   1.2977%  (var: 4.85%)
per-LLC mask (u8 operation; no LOCK prefix):   0.4930%  (var: 0.83%)
```

效果：nohz 掩码转换后整机性能持平、平均略好 ~1–2%；最坏场景（每次上下文切换一个事件）分布式更新代价为全局掩码的 **1/100**。未提供 nohz-heavy RT 场景的端到端延迟数据。

lore：RFC v3 cover

task enqueue/dequeue tracepoints（Nam Cao / Gabriele Monaco，Red Hat）

15 补丁系列的 01/15 · **[under review · 卡在动机说明]**

背景：现有 tracepoint 覆盖事件语义（wakeup/switch/migrate），缺「任务进出 runqueue」的**调度类无关**观察点（含 `__block_task()` 这类不走 `dequeue_task()` 的路径），只能靠 kprobe。为 RV（runtime verification）monitor 服务。

方案（3 文件 +21/−1）（`include/trace/events/sched.h`）

```
+DECLARE_TRACE(sched_enqueue,
+   TP_PROTO(struct task_struct *tsk, int cpu),
+   TP_ARGS(tsk, cpu));
+DECLARE_TRACE(sched_dequeue,
+   TP_PROTO(struct task_struct *tsk, int cpu),
+   TP_ARGS(tsk, cpu));
/* enqueue_task(): !(flags & ENQUEUE_DELAYED) 时打点;
   dequeue_task(): !(flags & DEQUEUE_SLEEP) 时打点; 先 trace_*_enabled() 控开销 */
```

进展：补丁本体早获 K Prateek RB，但已辗转三个宿主系列（20 补丁 RFC → RV 10 补丁 → 15 补丁），Peter 10-01 明确表示「忘了把其余补丁发给我、changelog 没说动机，As is I’m clueless as to why we want this」——合入卡在系列叙事而非代码。

idle SMT priority for asymmetric capacity（Mete Durlu，IBM/s390，RFC）

[rfc · 前提被质疑] · 日报 sched-20261008-001 → 20261014-014

背景：commit `25a32e400a14`（prefer fully-idle SMT cores）打破了 s390 依赖的行为——s390 的 capacity 是 hypervisor 分配的 entitlement（垂直极化），整机接近满载时**忙碌高容量核的空闲 SMT 兄弟应胜过完全空闲的低容量核**（聚合到高容量核让低容量核更久空闲、减少对邻分区噪声）。

方案演进：v1 为 `SCHED_IDLE_SMT_PRIO` 配置 + `arch_needs_idle_smt_prio()` 架构钩子 → 评审中转向扩展 `cpu_preferred_mask` 做 **soft preference 三档**（steal time==0 维持现状 / 低于阈值先填满 entitled 核 SMT 兄弟再外溢 / 超阈值收缩 preferred mask）。

效果：**无任何 benchmark 数据**；Shrikanth 的质疑未闭环——配好 steal governor 阈值后 preferred mask 里的低容量核本应很少，要求先用合理阈值实测「问题现实里是否成立」。RFC 阶段，观察即可。

附 · 正确性修复简报

TIF_NEED_RESCHED 置位顺序 [merged · 53bc5c556b82]

trace 点在 `TIF_NEED_RESCHED` 置位前触发，可被 BPF 程序递归重入直至内核栈溢出；新顺序在远程 CPU 分支还会破坏 RV nrp monitor 状态机（300 次 sched messaging 即违规）。

cputime idle 双计 [high · Frederic RB]

`cf6444c3e1bb` 统一记账后 dyntick-idle 与重启 tick 首周期重叠双计。

`SPARC idle/墙钟 1.44× → 1.0000 · KVM 1.53× → 0.998`

enqueue_task_fair 死代码回归 [high]

已合入的 `819224e506bc` 在 CFS_BANDWIDTH 下 throttle→dequeue→unthrottle→enqueue 命中被删分支触发 WARNING。Peter 修复方向：

```
+   if (cfs_rq->curr != se)
        __enqueue_entity(cfs_rq, se);
```

dl-server nohz_full 感知 [v3]

tick 启停改由 `inc/dec_dl_tasks()` 驱动；v2 曾把隔离核变成满 CONFIG_HZ tick，v3 附复现器与 running_bw 下溢 splats。

irq 线程热插拔竞态 [v2]

syzbot 报出：IRQ 线程迁到「当时在线但马上掉线」的 CPU，`__migrate_task()` 被拒后任务与 cpus_mask 失配；Juri 投票 `cpus_read_lock` 方案，Peter 要求完整故障图。

PSI trigger 销毁去 RCU 等待 [new]

Meta 生产环境：`synchronize_rcu()` 在持有 kernfs `node_mutex` 时阻塞文件关闭/cgroup 移除等用户；逐 commit 论证该等待已因历次重构失去必要性。

wake-affine 保持（非 SMT） [v6]

非 SMT 系统 narrow reciprocal `WF_SYNC` 唤醒保住 wake-affine 选中的 waker CPU。

`80 核 Altra pipe: 3.704 → 2.267 usec/op (+38.8%)`

iowait 记账 + nr_iowait_cpu 重构 [方案被否]

Wei Gao 修 liburing iowait.t 失败：`kcpustat_irq_enter()` 检查 `nr_iowait_cpu()` 提前结算——Peter 否决：

> "Using nr_iowait_cpu() is fundamentally never a fix. It is rolling a dice and liking random numbers."
> — Peter Zijlstra

衍生：Christian Loehle 的 `nr_iowait_cpu() → bool cpu_has_iowait()` 重构（附 Coccinelle recipe）。

deadline schedstats 恒零 [new]

`update_stats_dequeue_dl()` 唯一调用点走 throttle 路径且 flags=0，SCHED_DEADLINE 任务 sleep/block 统计全为 0：

```
+   if (on_dl_rq(&p->dl))
+       update_stats_dequeue_dl(&rq->dl, &p->dl, flags);
    dequeue_dl_entity(&p->dl, flags);
```

其他小修复

loadavg `calc_load_idx` 加 READ_ONCE/WRITE_ONCE；autogroup nice rate-limit 串行化（v2 改原子）；topology 分配失败释放 NUMA masks；sched stats 修正迁移 `sched_delayed` 任务的 run_delay 双计；wait 路径失败注册不改 entry。

结 · 收尾观察

- **节奏**：10 月是 7.3 发版周期 + 7.4 合入窗口开启的节点——约 20 个系列已进 tip / 主线（sched_ext 修复批、lost wakeup、TIF_NEED_RESCHED 顺序等），大量正确性修复集中收取。

- **高概率跟进合入**：task_h_load（全量 RB）、partial LLC utilization（双 RB + stable 候选）、cputime idle 双计（RB 待收取）、asym packing honor（RB + stable）。
- **值得持续跟踪**（cpuset/cgroup 视角）：cache-aware 打磨潮（放置策略与 LLC 容量判断）、NUMA balancing 粒度控制的接口之争（prctl vs BPF-cgroup）、SBM（nohz/cpumask 基础设施，LPC’26 有专场）。
- **普遍短板**：多数特性系列仍只有单一平台数据、无第三方复现——Gregory Price 在 NUMA 0/6 上的「要用户可见影响数据」是本月 review 的代表性姿态。

数据来源：`sched/2026/10/` 日报归档（2026-10-01 ~ 2026-10-14，187 篇）· 生成于 2026-10-15 · 返回日报首页 · GitHub 仓库